Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Hafta 9: Çok Kanallı Programlama

Yazılım Geliştirme II – Çalışma Kağıdı

Ad Soyad: _____	**Numara:** _____	**Tarih:** _____

Yönergeler

- Bu çalışma kağıdı **5 bölümden** oluşmaktadır.
- Çoktan seçmeli sorularda doğru seçeneği **daire içine alın**.
- Boşluk doldurma sorularında cevabınızı **çizgi üzerine** yazın.
- Doğru/yanlış sorularında **D** veya **Y** yazın.
- Süre: **40 dakika**
- Toplam: **60 puan**

Bölüm A: Çoktan Seçmeli Sorular

1. Bir **process** (süreç) ile bir **thread** (iş parçacığı) arasındaki temel fark nedir? **2 puan**
   A) Thread'ler ayrı adres alanına sahiptir, process'ler aynı alanı paylaşır
   B) Process'ler ayrı adres alanına sahiptir, thread'ler aynı process'in adres alanını paylaşır
   C) Thread'ler yalnızca tek çekirdekli sistemlerde çalışır
   D) Process'ler arası iletişim thread'lere göre daha hızlıdır

2. Aşağıdakilerden hangisi OpenMP'de bir döngüyü paralel çalıştırmak için kullanılan pragma direktifidir? **2 puan**
   A) `#pragma omp thread for`
   B) `#pragma omp parallel for`
   C) `#pragma omp distribute loop`
   D) `#pragma omp concurrent for`

3. `std::thread` nesnesi oluşturulduktan sonra ne `join()` ne de `detach()` çağrılmazsa ne olur? **2 puan**
   A) Thread arka planda sessizce çalışmaya devam eder
   B) Derleyici hatası oluşur
   C) `std::thread` yıkıcısı `std::terminate` çağırır
   D) Thread otomatik olarak join edilir

4. `std::lock_guard` ile `std::unique_lock` arasındaki temel fark nedir? **2 puan**
   A) `lock_guard` daha hızlıdır ancak thread-safe değildir
   B) `unique_lock` kilidi erteleyebilir, manuel açıp kapatabilir; `lock_guard` yalnızca RAII kilitleme yapar
   C) `lock_guard` birden fazla mutex'i aynı anda kilitleyebilir
   D) Aralarında hiçbir fark yoktur, sadece isim farklıdır

5. Aşağıdakilerden hangisi `std::atomic<int>` üzerinde **geçerli** bir işlem değildir? **2 puan**
   A) `sayac.store(42)`

B) `sayac.fetch_add(1)`

C) `sayac.lock()`

D) `sayac.load()`

Bölüm B: Boşluk Doldurma

Aşağıdaki cümlelerde boş bırakılan yerleri uygun ifadelerle doldurun.

1. Birden fazla thread'in aynı değişkene senkronizasyonsuz erişip en az birinin yazma yapması durumuna _____ denir ve bu C++ standardına göre tanımsız davranıştır. **2 puan**
2. `std::thread` nesnesine parametre geçirirken referans göndermek için `std::`_____`(degisken)` sarmalayıcısı kullanılır. **2 puan**
3. Bir mutex'i RAII ilkesiyle otomatik olarak kilitleyen ve kapsam sonunda serbest bırakan sınıf `std::`_____ olarak yazılır. **2 puan**
4. İki veya daha fazla thread'in birbirinin tuttuğu kilidi bekleyerek sonsuza dek bloke olması durumuna _____ denir. **2 puan**
5. `std::atomic<bool>` değişkeninin değerini okumak için _____`()` metodu, yazmak için _____`(d` metodu kullanılır. **2 puan**

Bölüm C: Doğru / Yanlış

Aşağıdaki ifadelerin doğru mu yanlış mı olduğunu belirtin.

1. OpenMP, derleme zamanında `-fopenmp` bayrağı verilmezse pragma direktifleri yorum olarak değerlendirilir ve program tek kanallı çalışır. **2 puan**
2. `std::thread::detach()` çağrıldıktan sonra thread nesnesi üzerinden `join()` çağrılabilir. **2 puan**
3. `std::lock_guard` kilidi kapsamdan çıkmadan önce manuel olarak serbest bırakmaya izin verir. **2 puan**
4. `std::atomic<int>` üzerindeki `++` operatörü atomik olarak çalışır. **2 puan**
5. Deadlock'tan kaçınmanın bir yolu, tüm thread'lerin mutex'leri aynı sırayla kilitlemesini sağlamaktır. **2 puan**

Bölüm D: Kod Okuma

1. Aşağıdaki kodun çıktısında `sayac` değeri her zaman `200000` olur mu? Neden? **10 puan**

```
#include <iostream>
#include <thread>

int sayac = 0;

void arttir() {
    for (int i = 0; i < 100000; ++i)
        ++sayac;
```

```
}

int main() {
    std::thread t1(arttir);
    std::thread t2(arttir);
    t1.join();
    t2.join();
    std::cout << sayac << '\n';
}
```

2. Aşağıdaki kodda bir deadlock riski var mıdır? Varsa hangi senaryoda oluşur, nasıl düzeltilir? **10 puan**

```
std::mutex m1, m2;

void gorevA() {
    std::lock_guard<std::mutex> k1(m1);
    std::lock_guard<std::mutex> k2(m2);
    // ... islem ...
}
void gorevB() {
    std::lock_guard<std::mutex> k1(m2);
    std::lock_guard<std::mutex> k2(m1);
    // ... islem ...
}
```

Bölüm E: Kod Yazma

1. `std::atomic<int>` kullanarak iki thread'in güvenli şekilde bir sayacı toplam 1 000 000 kez artırmasını sağlayan programı yazın. Her thread 500 000 artırma yapacak ve `main` sonunda sonucu yazdıracaktır. **10 puan**

Başarılar!